

Lecture 2: LU Factorization: Algorithm and Existence

Math 5610/6610

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Today's Goals

- Work a complete 4×4 LU factorization by hand

Today's Goals

- Work a complete 4×4 LU factorization by hand
- State the LU algorithm in MATLAB-style index notation

Today's Goals

- Work a complete 4×4 LU factorization by hand
- State the LU algorithm in MATLAB-style index notation
- Derive the operation count $\approx \frac{2n^3}{3}$ flops

Today's Goals

- Work a complete 4×4 LU factorization by hand
- State the LU algorithm in MATLAB-style index notation
- Derive the operation count $\approx \frac{2n^3}{3}$ flops
- See why LU without pivoting can be catastrophically unstable

LU Factorization (Continued) I

Example.

$$A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}$$

$$L_1 A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -4 & 0 & 1 & 0 \\ -3 & 0 & 0 & 1 \end{bmatrix} A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 3 & 5 & 5 \\ 0 & 4 & 6 & 8 \end{bmatrix}$$

$$L_2 L_1 A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -3 & 1 & 0 \\ 0 & -4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 3 & 5 & 5 \\ 0 & 4 & 6 & 8 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 2 & 2 \\ 0 & 0 & 2 & 4 \end{bmatrix}$$

$$L_3 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 \end{bmatrix}$$

LU Factorization (Continued) II

$$L_3 L_2 L_1 A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 2 \end{bmatrix} = U \quad \text{upper-triangular}$$

$$A = LU$$

$$L = L_1^{-1} L_2^{-1} L_3^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 4 & 0 & 1 & 0 \\ 3 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 3 & 1 & 0 \\ 0 & 4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 4 & 3 & 1 & 0 \\ 3 & 4 & 1 & 1 \end{bmatrix}$$

unit lower-triangular

Remark (*Convention (Matlab Notation)*).

$$A = [a_{jk}]_{j,k=1,2,\dots,n} \in \mathbb{C}^{n \times n}$$

$$a_{j,k:n} = [a_{j,k} \ a_{j,k+1} \ \cdots \ a_{j,n}]$$

LU Factorization (Continued) III

Remark (Algorithm: LU Factorization without pivoting). Input: $A \in \mathbb{C}^{n \times n}$

Output: L, U such that $A = LU$

Set $U = A, L = I$ ($n \times n$ identity matrix).

- for $k = 1, 2, \dots, n - 1$
 - for $j = k + 1, k + 2, \dots, n$
 - * $l_{j,k} = u_{j,k}/u_{k,k}$ (potential problem)
 - * $u_{j,k:n} = u_{j,k:n} - l_{j,k}u_{k,k:n}$
 - end
- end

Output: $L, U \in \mathbb{C}^{n \times n}$ such that $A = LU$

Remark (Operation Count). Additions:

$$\sum_{k=1}^{n-1} (n-k)(n-k+1) = \sum_{l=1}^{n-1} l(l+1) \approx \frac{n^3}{3} \quad (\text{where } l = n - k)$$

$$\text{Multiplications: } \approx \frac{n^3}{3}$$

Divisions: $\approx n^2 \Rightarrow$ ignore (because it is a lower order term)

LU Factorization (Continued) IV

Total Work: $\approx \frac{2n^3}{3}$ flops

Definition (Flop). A *flop* is a floating-point operation: addition, subtraction, multiplication, division, square root.

Remark (Solution of $Ax = b$).

$$\underbrace{A}_{=LU} x = b$$

$$L \underbrace{Ux}_{=y} = b$$

$$Ly = b \tag{1}$$

$$Ux = y \tag{2}$$

$Ly = b$ is easily solved via forward substitution. $Ux = y$ is easily solved by backward substitution.

One triangular solve requires $\approx n^2$ flops.

Problems with LU Factorization without Pivoting I

LU factorization without pivoting is unstable!

Example (*Error from LU Factorization without Pivoting*).

$$A = \begin{bmatrix} 10^{-20} & 1 \\ 1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

With exact arithmetic:

$$L = \begin{bmatrix} 1 & 0 \\ 10^{20} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 10^{-20} & 1 \\ 0 & 1 - 10^{20} \end{bmatrix}$$

Now solve a system:

$$Ly = b \quad y = \begin{bmatrix} 1 \\ -10^{20} \end{bmatrix}$$

$$Ux = y \quad x = \frac{1}{1 - 10^{-20}} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \approx \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

In a computer:

Problems with LU Factorization without Pivoting II

Floating point arithmetic does everything accurately up to ≈ 16 digits.

$$\tilde{L} = L = \begin{bmatrix} 1 & 0 \\ 10^{20} & 1 \end{bmatrix}, \quad \tilde{U} = \begin{bmatrix} 10^{-20} & 1 \\ 0 & -10^{20} \end{bmatrix}$$

$$\tilde{L}\tilde{U} = \begin{bmatrix} 10^{-20} & 1 \\ 1 & 0 \end{bmatrix} \neq A$$

If you use this to compute the answer, you will get a bogus answer.

Filling in the Details: The Operation Count I

At elimination step k we update rows $j = k + 1, \dots, n$ (that is $n - k$ rows), and in each row the entries in columns $k, \dots, n$ (that is $n - k + 1$ entries). Each updated entry costs one multiplication and one subtraction:

$$u_{j,k:n} = u_{j,k:n} - l_{j,k} u_{k,k:n}.$$

So the number of additions (and likewise multiplications) is

$$\sum_{k=1}^{n-1} (n-k)(n-k+1) \stackrel{l:=n-k}{=} \sum_{l=1}^{n-1} l(l+1) = \underbrace{\sum_{l=1}^{n-1} l^2}_{\approx n^3/3} + \underbrace{\sum_{l=1}^{n-1} l}_{\approx n^2/2} \approx \frac{n^3}{3}.$$

(Recall $\sum_{l=1}^m l^2 = \frac{m(m+1)(2m+1)}{6} \approx \frac{m^3}{3}$.)

Filling in the Details: The Operation Count II

Adding up:

$$\underbrace{\frac{n^3}{3}}_{\text{additions}} + \underbrace{\frac{n^3}{3}}_{\text{multiplications}} + \underbrace{O(n^2)}_{\text{divisions } l_{jk}} \approx \frac{2n^3}{3} \text{ flops.}$$

The divisions ($n - k$ per step, $\approx n^2/2$ total) are a lower-order term, so we drop them.

Consequence for $Ax = b$: factor once ($\frac{2n^3}{3}$), then each right-hand side costs only two triangular solves ($2n^2$). Solving with a second right-hand side is nearly free compared to refactoring.

Reading the 10^{-20} Example Correctly

What exactly went wrong? Track the sizes:

$$l_{21} = \frac{1}{10^{-20}} = 10^{20}, \quad u_{22} = 1 - 10^{20}.$$

In double precision (≈ 16 digits), $1 - 10^{20}$ rounds to -10^{20} : the “1” is annihilated. The stored factors then satisfy

$$\tilde{L}\tilde{U} = \begin{bmatrix} 10^{-20} & 1 \\ 1 & 0 \end{bmatrix},$$

which differs from A in the $(2, 2)$ entry by 1 – a 100% error in that entry, even though *each individual rounding was correct to 16 digits*.

Moral: the algorithm created huge intermediate quantities (10^{20}) not present in the original problem. Information about A was destroyed on the way. Note A itself is perfectly well-conditioned here – the *algorithm*, not the problem, is at fault. Pivoting (next lecture) keeps all multipliers $|l_{jk}| \leq 1$ and repairs this.

Looking Ahead

- **Next lecture:** Pivoting and $PA = LU$